

THE AUTOMAT HUB — PLAIN ENGLISH SETUP GUIDE

No technical experience assumed. Every step explained.

WHAT YOU HAVE BUILT

Think of the system in three parts:

1. **THE BACKEND** — The invisible engine that runs on a server. It stores data, issues Digital Condition Passports, manages escrow payments, and powers the live fleet tracking.
 2. **THE FRONTEND** — The web pages users see. Admin dashboard, fleet map, DCP verification page, inspector checklist app.
 3. **THE MOBILE APP** — A phone app (Android + iPhone) that connects to OBD-II adapters in vehicles, sends hourly health scans, and shows vehicle alerts to owners.
-

PART 1 — GETTING YOUR COMPUTER READY

Step 1 — Install four programs

Install these on your laptop. Click each link, download, and install normally.

A — Python 3.12 (the programming language) Download: <https://python.org/downloads>
WINDOWS USERS: During installation, tick "Add Python to PATH" Test it worked: open a terminal and type: `python --version` You should see: Python 3.12.x

B — PostgreSQL (the database) Download: <https://postgresql.org/download> During installation it will ask for a password. WRITE THAT PASSWORD DOWN. You will need it.

C — Redis (memory store for live tracking) Mac: open terminal and type: `brew install redis`
Windows: download from <https://github.com/microsoftarchive/redis/releases> Linux: type: `sudo apt install redis-server`

D — VSCode (the code editor) Download: <https://code.visualstudio.com>

Step 2 — Open the project

1. Unzip the `automat_hub.zip` file to your Desktop

2. Open VSCode
 3. Click File → Open Folder → choose the automat_hub folder
 4. You will see all files in the left panel
-

Step 3 — Open the terminal inside VSCode

Press: Ctrl + backtick (the key above Tab on your keyboard)

A terminal panel opens at the bottom of VSCode.

This is where you type all commands in this guide.

Step 4 — Create a virtual environment

A virtual environment is a private space for your project's code.

Type this in the terminal:

On Mac or Linux:

```
python -m venv venv  
source venv/bin/activate
```

On Windows:

```
python -m venv venv  
venv\Scripts\activate
```

You know it worked when you see (venv) at the start of your terminal line.

Every time you open a new terminal, run the activate command again.

Step 5 — Install all code libraries

```
pip install -r requirements.txt
```

Wait 3-5 minutes for this to finish.

Step 6 — Create your settings file

1. Find the file called .env.example in the project folder
2. Make a copy of it and rename the copy to exactly: .env (just .env — nothing else, no .txt at the end)

3. Open the .env file in VSCode
 4. Fill in these three values to start: SECRET_KEY — generate one by typing in terminal:
`python -c "import secrets; print(secrets.token_hex(32))"` Copy the output and paste it as the value. API_KEY — generate one by typing: `python -c "import secrets; print(secrets.token_hex(16))"` Copy and paste as the value. DATABASE_URL — replace YOUR_DB_PASSWORD with the password you wrote down when installing PostgreSQL. It should look like:
`DATABASE_URL=postgresql+asyncpg://automat_api:YourPasswordHere@localhost:5432/automat_hub`
-

Step 7 — Create the database

Type this in your terminal:

```
psql -U postgres
```

It will ask for your PostgreSQL password. Type it and press Enter.

You are now in the database. Type these commands one at a time:

```
CREATE DATABASE automat_hub;
```

Then replace YourPasswordHere with the same password you put in DATABASE_URL:

```
CREATE USER automat_api WITH PASSWORD 'YourPasswordHere';  
GRANT ALL PRIVILEGES ON DATABASE automat_hub TO automat_api;  
\q
```

Now create the database tables. Type:

```
psql -U postgres -d automat_hub -f database/migrations/001_create_dcp_tables.sql  
psql -U postgres -d automat_hub -f  
database/migrations/002_create_escrow_tables.sql
```

Step 8 — Start Redis

Mac: brew services start redis

Windows: Redis starts automatically as a Windows service after installation

Linux: sudo systemctl start redis

Step 9 — Run the application

```
uvicorn backend.main:app --reload
```

You should see:

THE AUTOMAT HUB — TRUST PROTOCOL

Environment: development

Open your browser and go to: <http://localhost:8000/docs>

You will see all the API endpoints. Your system is working.

Step 10 — Open the web pages

The frontend pages are HTML files. Open them in your browser:

- Admin Panel: double-click frontend/admin/index.html
- Fleet Map: double-click frontend/fleet/dashboard.html
- DCP Verify: double-click frontend/pages/verify.html
- Inspector: double-click frontend/pages/inspector.html

When testing locally they talk to <http://localhost:8000> automatically.

PART 2 — GETTING YOUR EXTERNAL ACCOUNTS

You need accounts with these services to enable payments, SMS, and maps.

Paystack — for receiving payments

1. Go to: <https://paystack.com> → Sign Up
2. Verify your business with your CAC number
3. Go to: <https://dashboard.paystack.com/#/settings/developer>
4. Copy your Secret Key and Public Key

5. Paste into your .env file: `PAYSTACK_SECRET_KEY=sk_live_XXXXXXXXXX`
`PAYSTACK_PUBLIC_KEY=pk_live_XXXXXXXXXX`

Use `sk_test_` keys while testing. Switch to `sk_live_` when real customers sign up.

Twilio — for SMS alerts

1. Go to: <https://twilio.com> → Start for Free
 2. Verify your phone number
 3. Go to: <https://console.twilio.com>
 4. Copy your Account SID and Auth Token
 5. Buy a phone number: Phone Numbers → Buy a number
 6. Paste into .env: `TWILIO_ACCOUNT_SID=ACXXXXXXXXXX`
`TWILIO_AUTH_TOKEN=XXXXXXXXXX` `TWILIO_PHONE_NUMBER=+1234567890`
-

AWS S3 — for photo storage

Stores inspection photos and QR codes.

1. Go to: <https://aws.amazon.com> → Create account
 2. Search for IAM → Users → Create User
 3. Name it: automat-hub-app
 4. Attach permission: AmazonS3FullAccess
 5. Go to Security Credentials → Create Access Key
 6. Copy the Access Key ID and Secret Access Key
 7. Go to S3 → Create Bucket named: automat-hub-production Region: eu-west-1 (Ireland)
 8. Paste into .env: `AWS_ACCESS_KEY_ID=AKIAXXXXXXXXXX`
`AWS_SECRET_ACCESS_KEY=XXXXXXXXXX` `AWS_BUCKET_NAME=automat-hub-production` `AWS_REGION=eu-west-1`
-

Firebase — for phone notifications

1. Go to: <https://console.firebase.google.com>
2. Create a project called: automat-hub

3. Go to Project Settings → Service Accounts
 4. Click Generate New Private Key → a JSON file downloads
 5. Open the JSON file and copy values into .env: FIREBASE_PROJECT_ID=automat-hub
FIREBASE_PRIVATE_KEY_ID=value from JSON FIREBASE_PRIVATE_KEY="the long key from JSON" FIREBASE_CLIENT_EMAIL=email from JSON
FIREBASE_CLIENT_ID=client_id from JSON
-

Google Maps — for fleet tracking map

1. Go to: <https://console.cloud.google.com>
 2. Create a new project
 3. Go to APIs → Library → enable Maps JavaScript API
 4. Go to APIs → Credentials → Create API Key
 5. Copy the key into:
 - .env file: GOOGLE_MAPS_API_KEY=AIzaSyXXXXXX
 - frontend/fleet/dashboard.html: find YOUR_GOOGLE_MAPS_KEY and replace it
-

SendGrid — for sending emails

1. Go to: <https://sendgrid.com> → Sign Up (free plan works)
 2. Go to Settings → API Keys → Create API Key → Full Access
 3. Paste into .env: SENDGRID_API_KEY=SG.xxxxxxxxxxx
FROM_EMAIL=noreply@automatcorp.org.ng FROM_NAME=The Automat Hub
-

PART 3 — PUTTING IT LIVE ON A REAL SERVER

Step 1 — Get a server

1. Go to: <https://digitalocean.com> → Create account
2. Click Create → Droplets
3. Choose:
 - Ubuntu 24.04 LTS

- Basic plan, 2GB RAM (\$12/month)
- Region: London (LON1) — fastest from Nigeria

4. Click Create Droplet

5. Write down the IP address shown (looks like: 167.99.87.123)

Step 2 — Connect to your server

In your terminal on your laptop, type (replace with your actual IP):

```
ssh root@167.99.87.123
```

Type yes when asked, then enter your password.

You are now controlling your server remotely.

Step 3 — Install software on the server

Copy and paste this whole block into the server terminal:

```
sudo apt update && sudo apt upgrade -y
sudo apt install python3.12 python3-pip python3.12-venv postgresql redis-server
nginx -y
```

Step 4 — Upload your project files

In a NEW terminal on your laptop (not the server), type:

```
scp -r /path/to/automat_hub root@167.99.87.123:/home/automat_hub
```

Replace /path/to/automat_hub with where the folder actually is on your laptop.

For example if it is on your Desktop: /Users/yourname/Desktop/automat_hub

Step 5 — Set up the project on the server

Back in the server terminal:

```
cd /home/automat_hub
python3.12 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Create the .env file with production values:

```
nano .env
```

Paste all your settings (same as your local .env but with production keys).

Press Ctrl+X then Y then Enter to save.

Step 6 — Set up the database on the server

```
sudo -u postgres psql
```

Inside the database shell, type each line and press Enter:

```
CREATE DATABASE automat_hub;
CREATE USER automat_api WITH PASSWORD 'your-strong-password';
GRANT ALL PRIVILEGES ON DATABASE automat_hub TO automat_api;
\q
```

Run the migrations:

```
psql -U postgres -d automat_hub -f database/migrations/001_create_dcp_tables.sql
psql -U postgres -d automat_hub -f
database/migrations/002_create_escrow_tables.sql
```

Step 7 — Point your domain to the server

Go to wherever you registered automatcorp.org.ng (Namecheap, GoDaddy etc).

Find DNS Settings and add:

Type: A	Host: @	Value: 167.99.87.123
Type: A	Host: www	Value: 167.99.87.123

Wait 15-30 minutes for this to take effect.

Step 8 — Set up Nginx (the web gateway)

```
sudo nano /etc/nginx/sites-available/automat
```

Paste this (replace automatcorp.org.ng with your domain):

```
server {
    listen 80;
    server_name automatcorp.org.ng www.automatcorp.org.ng;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}
```

Save (Ctrl+X, Y, Enter), then run:

```
sudo ln -s /etc/nginx/sites-available/automat /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

Step 9 — Add HTTPS (the padlock)

Free and takes 2 minutes:

```
sudo apt install certbot python3-certbot-nginx -y
sudo certbot --nginx -d automatcorp.org.ng -d www.automatcorp.org.ng
```

Follow the prompts. Certbot handles everything.

Step 10 — Keep the app running permanently

```
sudo nano /etc/systemd/system/automat.service
```

Paste:

```
[Unit]
Description=The Automat Hub API
After=network.target
[Service]
User=root
WorkingDirectory=/home/automat_hub
Environment="PATH=/home/automat_hub/venv/bin"
ExecStart=/home/automat_hub/venv/bin/gunicorn backend.main:app -w 4 -k
uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000
Restart=always
[Install]
WantedBy=multi-user.target
```

Save, then:

```
sudo systemctl enable automat
sudo systemctl start automat
```

Check it is running:

```
sudo systemctl status automat
```

You should see green text saying "active (running)".

Test it: open <https://automatcorp.org.ng/docs> in your browser. If you see the API documentation, you are live.

Step 11 — Start the hourly scan scheduler

```
sudo nano /etc/systemd/system/automat-worker.service
```

Paste:

```
[Unit]
Description=Automat Hub Background Jobs
After=network.target redis.service
[Service]
User=root
WorkingDirectory=/home/automat_hub
Environment="PATH=/home/automat_hub/venv/bin"
ExecStart=/home/automat_hub/venv/bin/celery -A backend.tasks.scheduler worker --
beat -l info
Restart=always
[Install]
WantedBy=multi-user.target
```

Save, then:

```
sudo systemctl enable automat-worker
sudo systemctl start automat-worker
```

PART 4 — THE MOBILE APP

Step 1 — Install Node.js

Download from: <https://nodejs.org> (install the LTS version)

Step 2 — Install Expo in your terminal

```
npm install -g expo-cli eas-cli
```

Step 3 — Create the app

```
npx create-expo-app AutomatHubApp --template blank-typescript
cd AutomatHubApp
npm install @react-navigation/native @react-navigation/bottom-tabs expo-barcode-
```

```
scanner expo-camera expo-notifications expo-location expo-secure-store axios  
react-native-chart-kit
```

Step 4 — Connect it to your live server

In the file `src/services/api.ts`, find the line: `const API_URL = 'https://automatcorp.org.ng';`

This is already correct. It will talk to your live server.

Step 5 — Test on your phone

```
npx expo start
```

A QR code appears in the terminal.

Android: Install the Expo Go app from Play Store. Scan the QR code.

iPhone: Open the Camera app and point it at the QR code.

The app opens on your phone instantly.

Step 6 — Publish to Android

You need a Google Play Developer account first (\$25 one-time): <https://play.google.com/console>
→ Create Account

Then build and upload:

```
eas build --platform android --profile production
```

Wait 10-20 minutes. You get a download link for a .aab file.

Go to play.google.com/console → Create app → Release → Production

Upload the .aab file and submit for review.

Google reviews it in 3-7 days.

Step 7 — Publish to iPhone

You need an Apple Developer account first (\$99/year): <https://developer.apple.com> → Enroll

Then:

```
eas build --platform ios --profile production
eas submit --platform ios
```

Apple reviews the app in 1-3 days.

PART 5 — THE OBD-II ADAPTER

Every car made after 1996 has an OBD-II port under the dashboard (usually below and left of the steering wheel).

Buy one of these adapters:

- Search "ELM327 OBD2 Bluetooth" on Jumia
- Price: ₦8,000 to ₦15,000
- Make sure it says Bluetooth 4.0 or BLE

How it works:

1. Customer plugs the adapter into the car's OBD port
 2. Opens the Automat Hub app
 3. App finds the adapter via Bluetooth and pairs with it
 4. Every hour the app wakes up and reads fault codes
 5. Data is sent to your server automatically
 6. If faults are found the customer gets an instant notification
 7. The notification tells them the nearest registered workshop
-

TROUBLESHOOTING

App will not start:

Make sure .env file exists with DATABASE_URL, SECRET_KEY and API_KEY filled in.

Make sure you activated your virtual environment (you see (venv)).

Make sure PostgreSQL is running.

Database connection error:

Check the password in DATABASE_URL matches your PostgreSQL password.

Check PostgreSQL is running.

Check you created the automat_hub database.

401 errors (not authorized):

Your login token has expired after 8 hours. Log in again.

Payments not working:

Check you are using Paystack TEST keys while testing.

Only switch to LIVE keys when real customers are using the system.

SMS not delivering:

Check Twilio account has credit loaded.

Make sure phone numbers include country code (+234 for Nigeria).

Map not showing:

Check Google Maps API key is correct.

Make sure Maps JavaScript API is enabled in your Google Console.

Add your domain to the allowed domains in Google Console API key settings.

SUMMARY OF ALL KEYS NEEDED

Key	Where to get it	Time needed
SECRET_KEY	Run python command above	30 seconds
API_KEY	Run python command above	30 seconds
PAYSTACK_SECRET_KEY	dashboard.paystack.com	15 minutes
TWILIO_ACCOUNT_SID	console.twilio.com	15 minutes
AWS_ACCESS_KEY_ID	aws.amazon.com/iam	20 minutes
GOOGLE_MAPS_API_KEY	console.cloud.google.com	15 minutes
FIREBASE_PROJECT_ID	console.firebase.google.com	20 minutes
SENDGRID_API_KEY	sendgrid.com	10 minutes

Support: ceo@automatcorp.org.ng | +234 916 747 6422